

Contents

1	More on Data Passing	1
1.1	GET Method	1
1.1.1	Characteristics of GET:	1
1.1.2	Example Usage:	1
1.2	POST Method	2
1.2.1	Characteristics of POST:	2
1.2.2	Example Usage:	2
1.2.3	Conclusion	2

1 More on Data Passing

Passing data through URLs or forms is a fundamental aspect of web development with PHP. The two primary methods for this are GET and POST, each with its specific use cases and characteristics.

1.1 GET Method

The GET method sends data through the URL. It's primarily used for requesting data from a specific resource and is visible to the user in the browser's address bar.

1.1.1 Characteristics of GET:

- Data is appended to the URL as query string parameters.
- Suitable for non-sensitive data, as it's visible in the URL.
- Limited amount of data can be sent (depends on the browser's URL length limit).
- Can be bookmarked and cached.
- Faster compared to POST for small amounts of data.

1.1.2 Example Usage:

```
<!-- HTML Form using GET method -->
<form action="process.php" method="get">
    Name: <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

In PHP, you can access GET data using the `$_GET` superglobal array.

```
<?php
if (isset($_GET['name'])) {
    echo "Hello, " . htmlspecialchars($_GET['name']);
}
?>
```

1.2 POST Method

The POST method sends data through the HTTP request body. It's used for sending data to a server to create/update a resource and is not visible in the URL.

1.2.1 Characteristics of POST:

- Data is not displayed in the URL.
- Suitable for sending sensitive or large amounts of data.
- No size limitations for the amount of data (within practical limits).
- Cannot be bookmarked or cached.
- Slightly slower for small amounts of data due to overhead.

1.2.2 Example Usage:

```
<!-- HTML Form using POST method -->
<form action="process.php" method="post">
    Name: <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

In PHP, POST data is accessed using the `$_POST` superglobal array.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    echo "Hello, " . htmlspecialchars($_POST['name']);
}
?>
```

1.2.3 Conclusion

- Use **GET** for simple, non-secure data retrieval, especially when you want to be able to bookmark or share the URL.
- Use **POST** for submitting forms, especially when dealing with sensitive data or when the amount of data is considerable.

Both methods are integral to PHP web development, allowing for efficient data transfer between client and server. Understanding when and how to use each method is key to building robust and secure web applications.